Administrator

You own the deployment. This chapter is what only you can do, and the four things that will hurt if you do them carelessly.

> Read `00-orientation.md` first. You hold every permission in the system, including any added by a future release — so every other chapter in this manual also describes something you can do.

Settings

Settings sits apart from the operations navigation and has its own shell. The reason is stated in the product: *nothing here moves a shipment.* These screens configure the operator — who may sign in, what a branch invoices as, what the product is called.

SCREEN	WHAT IT IS FOR
Users & roles	Who can sign in, and what they hold in each branch
Branches	Currency, invoice series, working calendar, per entity
Audit log	Every write, by whom, from where
Vendor access	What your supplier's support can see, and for how long
Brand	Colours, name and mark, and the variable behind each

Reference data deliberately stayed in operations. Ports, charge types, milestone templates and exception codes are daily operational work, and putting them behind an administrative wall is how a system grows a free-text "Other" field that swallows the data it was meant to structure. Your operations manager owns them — see `02-operations-manager.md`.

Users and roles

Adding someone

The product is invitation-only. There is no sign-up form, so there is nothing to bootstrap from and nothing for a stranger to find.

Accounts are created from a shell on the server:

```
pnpm db:create-user --email ops@yourcompany.com --name "Operations" \
                    --role operations_manager
```

That writes, in one transaction, the four rows an account needs: the auth user, the hashed credential, the domain principal, and **a role per branch**. Without the branch role the account signs in and sees nothing — which is the most common way a new user's first day goes wrong.

By default it grants the role in every active branch. To scope it to one:

```
pnpm db:create-user --email kwi@yourcompany.com --name "Kuwait Desk" \
                    --role coordinator --branch KWI
```

The generated password prints once. Give it to the person directly. Do not email it.

Resetting a password

Re-run the same command for an existing email. It resets that password and changes nothing else. **This is the password-reset path** until a mail transport is configured, and it is why there is no "forgot password" link.

Roles are per branch

The same person can be a **coordinator in Kuwait and read-only in Dubai**. That is the intended shape, not a workaround — a person who covers another office during a holiday gets read access there and keeps their own role at home.

Every screen counts and shows only the branches a person holds a role in. There is no "all branches" toggle, and there should not be.

Two-factor

Administrator and Finance must enrol. They are the two roles that can change who has access and can issue money documents. Nobody can opt out, including you.

Choosing a role

Seven roles, described in full in `09-permissions-reference.md`. Three rules of thumb:

Grant the narrowest role that lets someone do their job. The customs broker role exists precisely so that a broker does not need a coordinator's access.

Do not put an operations person in the Finance role because they "help with invoices". The separation between issuing invoices and running operations is the one an auditor asks about first, and it is enforced rather than advised.

Read-only is a real answer. For an owner who wants to look, a new joiner in their first fortnight, or an accountant who needs to see and not touch.

Branches

Each branch carries **its own base currency, its own invoice series and its own working calendar**.

	WHY IT MATTERS
Currency	What the entity invoices in
Invoice series	Numbering is per branch, and gaps are questions at audit. Voided invoices keep their number for exactly this reason.
Working calendar	**Every SLA clock and ageing rule counts against this.** Kuwait, Qatar and the UAE share neither a weekend nor a holiday list.

Load next year's public holidays before January. If you do not, every deadline over the break is wrong, in the same direction, for every branch — and nobody notices until a customer does.

The audit log

Every write, recorded **by the database itself** rather than by the application — so a change that goes around the application is still recorded. When, what operation, which table, who, which branch, which record, from which address.

Nobody can edit or delete an entry. Update and delete are revoked from every role in the system, including yours and including your supplier's. This is not a policy; it is a permission that does not exist.

Read it when a figure is disputed or a change needs attributing. That is what it is for, and it is worth telling your team it exists — not as a threat, but because a team that knows changes are attributable argues less about what happened.

Vendor access — the one to understand properly

Support cannot see your data by default. There is no standing access, no support account with a permanent key, and no back door.

When your supplier needs to look at something, **you grant it**, from **Settings → Vendor access**:

FIELD	WHAT TO PUT
Engineer	The named person. Not a team.
Ticket or reason	e.g. `SUP-1042 — invoice total disputed on KWI-2026-01041`. Recorded against every session opened under the grant.
Expires after	A number of hours. Choose the smallest that is plausible.
Access	Read only, or read and write
Branches	Leave blank for the branches you hold. **Choosing fewer is always safer.**

Read-and-write requires typing `GRANT WRITE` to confirm. The friction is deliberate. Most support work is diagnosis and needs read access only — ask what they intend to change before you grant write, and if the answer is "nothing yet", grant read.

While a grant is live, **a banner sits at the top of every page** of your system saying so, for everyone. Nobody is unaware that an outsider is in.

You can **revoke it in one click**, at any time, without explanation.

Every grant and every session opened under it is recorded permanently, and *that record cannot be edited or deleted by anyone, including your supplier*. So the question "what did they look at" always has an answer.

Habits worth keeping:

- Grant hours, not days
- Grant read unless write has been justified in words you understood
- Grant the branch in question, not all of them

- Revoke when the call ends rather than waiting for expiry
- Read the vendor-access screen once a month even when nothing is open

---

Brand

Settings → Brand shows every colour, name and mark the product resolves at runtime — and the environment variable behind each one.

The whole product is re-skinned by setting variables. No rebuild, no code change:

```
BRAND_NAME_EN="Meridian Logistics" BRAND_MONOGRAM=M \
BRAND_PRIMARY_HUE=155 BRAND_ACCENT_HUE=25 BRAND_RADIUS=4
```

Both themes, every tonal ramp, every semantic colour and the chart series derive from those tokens. The screen renders the resolved values so the claim is inspectable rather than asserted.

Two things deliberately do not follow your brand, and should not be made to:

Status colours. Positive, caution, critical, info and muted are reserved for state. A board where "held" is calm because it matched a corporate palette is worse than useless. If your brand hue lands within about 15° of one of them, it will read as that status — move the brand hue, not the status colour.

Chart series. Four slots, assigned in order and never cycled. All four are validated for lightness, contrast against both surfaces, and separation under the two common forms of colour blindness, in both light and dark themes. Changing one means re-running that validation.

Bulk import

Network → Bulk import. Bringing existing data in from a spreadsheet, checked row by row before anything is written.

What can be imported

	NOTES
Customers	Both names, code, terms
Customer contacts	Attached to customers by code
Agents and hauliers	The partner register
Ports and places	
Rate cards	Buy — an agent tariff, needs the partner — or sell
Open jobs	For a mid-year cutover
Open receivables	For a mid-year cutover

How it works

1. **Choose what to import**, and the branch. Every row is created against that branch.

2. **Download the template** if you want the columns in the right order.
3. **Choose the file.** CSV or tab-separated, saved from Excel. The first row must be the headings.
4. **Check the file.** Columns are matched to your headings automatically; correct anything your agent named differently. Unmatched columns are listed as ignored so nothing is silently dropped.
5. **Read the result.** Rows read, rows accepted, and every problem with its line number. You can download the problem list.
6. **Import**, only if everything passed.

The rule that matters

A file with any errors imports nothing. Not the good rows — nothing.

That is deliberate and it is worth defending when somebody asks for a partial-import option. A half-imported customer list is worse than no import, because nobody can tell which half is in, and the second attempt creates duplicates of the first.

Rate cards will be rejected for overlapping an existing card on the same lane. That is the protection working — two overlapping cards on one lane are unrepresentable in this system, by design. Fix the validity dates on one of them.

Doing a real migration

Order matters, because later files refer to earlier ones:

1. Ports and places
2. Agents and hauliers
3. Customers
4. Customer contacts
5. Rate cards — sell first, then buy per partner
6. Open jobs
7. Open receivables

Import into one branch first and look at the result before running the rest. Ten minutes of checking here saves a day of unpicking, and there is no undo — the correction path is editing records, not re-running the file.

Operating the deployment

If your supplier operates the deployment, this section is theirs. If you operate it yourself, this is the whole of the job.

COMMAND	WHAT IT DOES
`pnpm db:migrate`	Apply schema migrations. Forward-only.
`pnpm db:seed`	Reference rows and the role/permission matrix
`pnpm db:create-user`	Provision an account that can sign in
`pnpm fleet:status`	What schema version each deployment is on, and whether it is reachable

COMMAND	WHAT IT DOES
`pnpm fleet:migrate`	Migrate across deployments, sequentially, with a report
`pnpm fleet:backup`	Dump every deployment
`pnpm fleet:restore-drill`	Dump, restore into a scratch database, verify, drop

`fleet:restore-drill` is the one to schedule. A backup nobody has restored is a hypothesis. The drill restores into a scratch database and checks the *content*, not the exit status — including that the security policies survived, because a restore that drops them produces a database that looks perfect and enforces nothing.

Dumps must be taken through the admin connection. The application account cannot read outside its own branches, so a dump taken as the application role is silently partial — the most dangerous artefact in this whole area, because it looks like a backup.

What only you can break

Granting the wrong role. Every other mistake in this manual is recoverable from the audit log. This one is somebody seeing what they should not, and it is not visible until it matters. Grant the narrowest role.

Granting standing vendor write access. Hours, not days. Read, not write, unless justified.

Forgetting the working calendar. Every SLA in the system counts against it. A missing holiday list is every deadline wrong in the same direction, silently.

An unverified backup. Run the restore drill on a schedule. Do not assume the dump is good because the command exited zero.

What is not in this release

A user-invitation screen	Accounts are created from a shell. It is also the password-reset path.
Self-service password reset	Needs a mail transport configured.
Editing the role matrix from the interface	The matrix is seeded and editable without a release, but not from a screen today.
Email and WhatsApp notifications	In delivery. Templates exist; the sending channels are being finished.
Scheduled reports	In delivery. Every screen exports today.